

МИНИСТЕРСТВО ОБРАЗОВАНИЯ КИРОВСКОЙ ОБЛАСТИ  
Кировское областное государственное профессиональное образовательное  
бюджетное учреждение  
«Слободской колледж педагогики и социальных отношений»

**ОТЧЕТ**

**по учебной практике**

**ПМ.02. Осуществление интеграции программных модулей**

Студент

Мурин Денис Валерьевич

Группа 23П-1

Специальность 09.02.07 Информационные  
системы и программирование

Руководитель практики от колледжа:

*Седов Алексей Сергеевич*

Руководитель практики от организации:

\_\_\_\_\_ *Седов Алексей Сергеевич*

подпись

УТВЕРЖДАЮ:

Директор

\_\_\_\_\_  
Наименование организации

\_\_\_\_\_  
\_\_\_\_\_

\_\_\_\_\_/\_\_\_\_\_

подпись

расшифровка

М. П.

2025-2026 уч. год

# Содержание

## Оглавление

- Введение
- Анализ предметной области
- Руководство оператора
- Работа в системе контроля версий
- Разработка программных модулей
  - База данных
  - API
  - Модуль технолог
  - Модуль лаборатории
  - Модуль аппаратчик
- Разработка тестовых наборов и тестовых сценариев
- Отладка программного модуля
- Заключение.

## Приложения

## Введение

Учебная практика по профессиональному модулю ПМ.02 «Осуществление интеграции программных модулей» проходила на базе КОГПОБУ «Слободской колледж педагогики и социальных отношений» с 6 мая по 17 мая 2026 года (согласно дневнику практики). Продолжительность практики составила 72 часа.

Цель практики: формирование практических навыков по интеграции программных модулей в единую систему, разработке требований, отладке и тестированию программного обеспечения.

## Задачи практики:

1. Проанализировать предметную область и разработать требования к программным модулям.
2. Разработать базу данных, API и три desktop-модуля (Технолог, Лаборатория, Аппаратчик).
3. Осуществить интеграцию модулей с использованием Git и GitHub.
4. Провести отладку и тестирование программных модулей.
5. Оформить отчётную документацию.

Объект исследования: процесс интеграции программных модулей на примере производственного предприятия (управление рецептурами, качеством продукции и технологическими процессами).

## Анализ предметной области

Предметная область – **производственное предприятие**, где выделяются три роли пользователей:

- Технолог – разрабатывает рецептуры продукции (состав, время изготовления, параметры).
- Лаборатория – проводит анализы готовых партий, вводит результаты контроля качества.
- Аппаратчик – выполняет производственные задания по утверждённым рецептам.

Требования к информационной системе:

- Централизованное хранение данных (единая база данных).
- Обмен данными между модулями через REST API.
- Независимость модулей (каждый может работать отдельно).
- Поддержка многопользовательского режима.

В ходе анализа были выявлены следующие сущности:

- Рецепты (id, название, состав, время, статус).
- Партии (id, id рецепта, дата запуска, статус, результат анализов).
- Пользователи (id, логин, роль, пароль).
- Результаты анализов (id, id партии, дата, показатели).

На основе этого анализа разработана ERD-диаграмма (представлена в приложении).

## Руководство оператора

Руководство оператора описывает работу каждого из трёх desktop-модулей. Подробная версия вынесена в отдельный документ (Руководство\_оператора.docx), здесь приведены основные моменты.

### Модуль «Технолог»

- **Назначение:** создание и редактирование рецептов.
- **Основные действия:** вход в систему, создание нового рецепта, просмотр списка рецептов, изменение статуса.
- **Пример:** после авторизации открывается форма со списком рецептов. Кнопка «Новый рецепт» открывает окно для ввода названия, состава и времени. После сохранения рецепт появляется в общем списке и становится доступен для лаборатории и аппаратчика.

### Модуль «Лаборатория»

- **Назначение:** ввод результатов анализов качества для произведённых партий.
- **Основные действия:** выбор партии, ввод показателей (влажность, плотность и др.), отправка результата.

### Модуль «Аппаратчик»

- **Назначение:** выполнение производственных заданий.
- **Основные действия:** просмотр активных заданий, запуск процесса, отметка о выполнении.

Взаимодействие модулей осуществляется через API: все модули отправляют HTTP-запросы к единому серверу, который работает с базой данных.

## Работа в системе контроля версий

В ходе практики для управления версиями программных модулей использовалась система Git и клиент GitHub Desktop. Репозиторий размещён по адресу: <https://github.com/Hefamr01/PM.02-Integration-of-software-modules>

### Выполненные действия:

1. Создан локальный репозиторий.
2. Подготовлена структура папок (документация, исходный код, скрипты БД, тесты, диаграммы).
3. Выполнены коммиты с осмысленными сообщениями (например, «Добавлен модуль Технолог», «Создан API», «Обновлены тест-кейсы»).
4. Репозиторий синхронизирован с удалённым сервером GitHub (публичный доступ).

5. Настроен файл .gitattributes для устранения проблем с окончаниями строк (CRLF/LF) при работе в разных ОС.
6. Добавлен файл .gitignore для исключения временных файлов Visual Studio (bin/, obj/, .vs/).
7. В результате все версии кода и документов сохранены, обеспечена возможность совместной работы (при необходимости) и контроля изменений.

## Разработка программных модулей

### База данных

Для хранения данных использовалась **СУБД PostgreSQL** (также поддерживается MS SQL Server). Скрипт создания базы данных и заполнения тестовыми данными находится в папке database/script.sql репозитория.

### Структура БД:

- Таблица users (id, login, password\_hash, role).
- Таблица recipes (id, name, composition, time\_min, status, technologist\_id).
- Таблица batches (id, recipe\_id, start\_date, status, operator\_id).
- Таблица analysis\_results (id, batch\_id, date, moisture, density, note).

Скрипт создаёт эти таблицы, а также добавляет несколько тестовых записей (администратор, два рецепта, партия).

### API

API разработано на ASP.NET Core (или аналогичном фреймворке C#). Коллекция запросов для тестирования в Postman сохранена в api/postman\_collection.json.

### Доступные эндпоинты:

- POST /api/auth – авторизация пользователя.
- GET /api/recipes – получение списка рецептов.
- POST /api/recipes – создание нового рецепта.
- GET /api/batches – список партий.
- POST /api/batches/{id}/analysis – добавление результатов анализа.
- PUT /api/batches/{id}/status – изменение статуса партии.

API взаимодействует с БД через Entity Framework Core. Все запросы возвращают ответы в формате JSON.

### Модуль «Технолог»

Desktop-приложение, реализованное на C# (Windows Forms). Исходный код находится в desktop-modules/TechModule.

Функции:

- Авторизация (роль «технолог»).
- Просмотр списка рецептов (загружается через API).
- Создание/редактирование рецепта (POST запрос к API).
- Удаление рецепта (при отсутствии связанных партий).

## Модуль «Лаборатории»

Расположен в desktop-modules/LabModule. Основной функционал:

- Авторизация роли «лаборант».
- Получение списка незавершённых партий (GET /api/batches?status=active).
- Ввод результатов анализа и отправка POST /api/batches/{id}/analysis.

## Модуль «Аппаратчик»

Расположен в desktop-modules/OperatorModule. Предоставляет:

- Авторизация роли «аппаратчик».
- Отображение заданий (GET /api/batches?status=assigned).
- Смена статуса задания на «выполнено» (PUT /api/batches/{id}/status).

## Разработка тестовых наборов и тестовых сценариев

В ходе практики разработаны тест-кейсы (хранятся в tests/test-cases.md). Примеры тестов:

| ID    | Название                               | Шаги                                                              | Ожидаемый результат                           |
|-------|----------------------------------------|-------------------------------------------------------------------|-----------------------------------------------|
| ТС-01 | Создание рецепта через модуль Технолог | 1. Войти как технолог. 2. Заполнить форму. 3. Нажать «Сохранить». | Рецепт появляется в списке и в БД.            |
| ТС-02 | Отправка результатов анализа           | 1. Войти в лабораторию. 2. Выбрать партию. 3. Ввести показатели.  | Статус партии меняется на «проанализировано». |
| ТС-03 | Авторизация с неверным паролем         | Ввести неверный пароль.                                           | Сообщение об ошибке, доступ не предоставлен.  |

Тестирование показало, что все модули успешно интегрированы, ошибки авторизации обрабатываются, данные корректно сохраняются.

## Отладка программного модуля

В процессе разработки и интеграции проводилась отладка с использованием средств Visual Studio (точки останова, просмотр переменных, окно Immediate) и логирования.

### Обнаруженные и исправленные дефекты:

1. **NullReferenceException** при загрузке списка рецептов в модуле Технолог, если API вернуло пустой ответ.

*Исправление:* добавлена проверка на null перед обработкой данных.

2. **Несоответствие формата даты между API** и модулем Лаборатории (ошибка при сохранении результата).

*Исправление:* унифицирован формат ISO 8601.

3. **Проблема с окончаниями строк (CRLF/LF)** при коммитах в Git.

*Исправление:* добавлен файл .gitattributes со строкой \* text=auto.

4. **API возвращал код 500 вместо 400** при передаче пустого тела запроса.

*Исправление:* добавлена валидация модели и обработка BadRequest.

5. **Модуль Аппаратчик не обновлял статус задания** после завершения из-за неверного SQL-запроса.

*Исправление:* скорректирован запрос и метод PUT в API.

**Результат отладки:** все программные модули работают стабильно, ошибки обработаны, исключения логируются.

## Заключение

В ходе учебной практики ПМ.02 были выполнены все поставленные задачи:

1. Проведён анализ предметной области, разработаны требования к программным модулям.
2. Создана база данных, разработаны REST API и три desktop-модуля (Технолог, Лаборатория, Аппаратчик).
3. Осуществлена интеграция модулей через API и систему контроля версий Git.
4. Проведены тестирование и отладка, устранены выявленные дефекты.
5. Оформлена полная документация (отчёт, дневник, аттестационный лист, пояснительная записка, руководство оператора, документ по отладке, презентация).

Все артефакты загружены в публичный репозиторий GitHub. Структура репозитория соответствует требованиям (отсутствие архивов, наличие оригиналов и PDF-копий документов, оформленный README).

Практика пройдена успешно, сформированы профессиональные компетенции ПК 2.1–2.5 и общие компетенции ОК 01–ОК 11.

## Приложения

### 1. Скриншоты интерфейсов модулей:

#### а) Приложение 1 – Главное окно модуля «Технолог» [Рисунок 1]

**Рецепты продукции**

| ID | Название     | Состав                   | Время (мин) |
|----|--------------|--------------------------|-------------|
| 1  | Пиво светлое | солод, хмель, вода       | 60          |
| 2  | Пиво тёмное  | солод карамельный, хмель | 75          |
| 3  | Квас         | ржаной солод, сахар      | 40          |
| 4  | Эль          | эльские дрожжи, солод    | 90          |

Поиск:

**Найти** **+ Новый рецепт** **Редактировать** **Удалить** **Обновить**

Рисунок 1 - Окно модуля «Технолог»

#### б) Приложение 3 – Окно модуля «Лаборатория» [Рисунок 2]

**Ввод результатов анализов**

Партия:  Влажность (%):  Плотность (\*Plato):  **Отправить результат**

| ID партии | Рецепт       | Статус          | Результат анализа       |
|-----------|--------------|-----------------|-------------------------|
| 101       | Пиво светлое | Ожидает анализа | —                       |
| 102       | Пиво тёмное  | Ожидает анализа | —                       |
| 103       | Квас         | Анализ выполнен | Влажн. 3.8%, Плотн. 8.5 |

Рисунок 2 - Окно модуля «Лаборатория»



с) Приложение 4 – Модуль «Аппаратчик» [Рисунок 3]

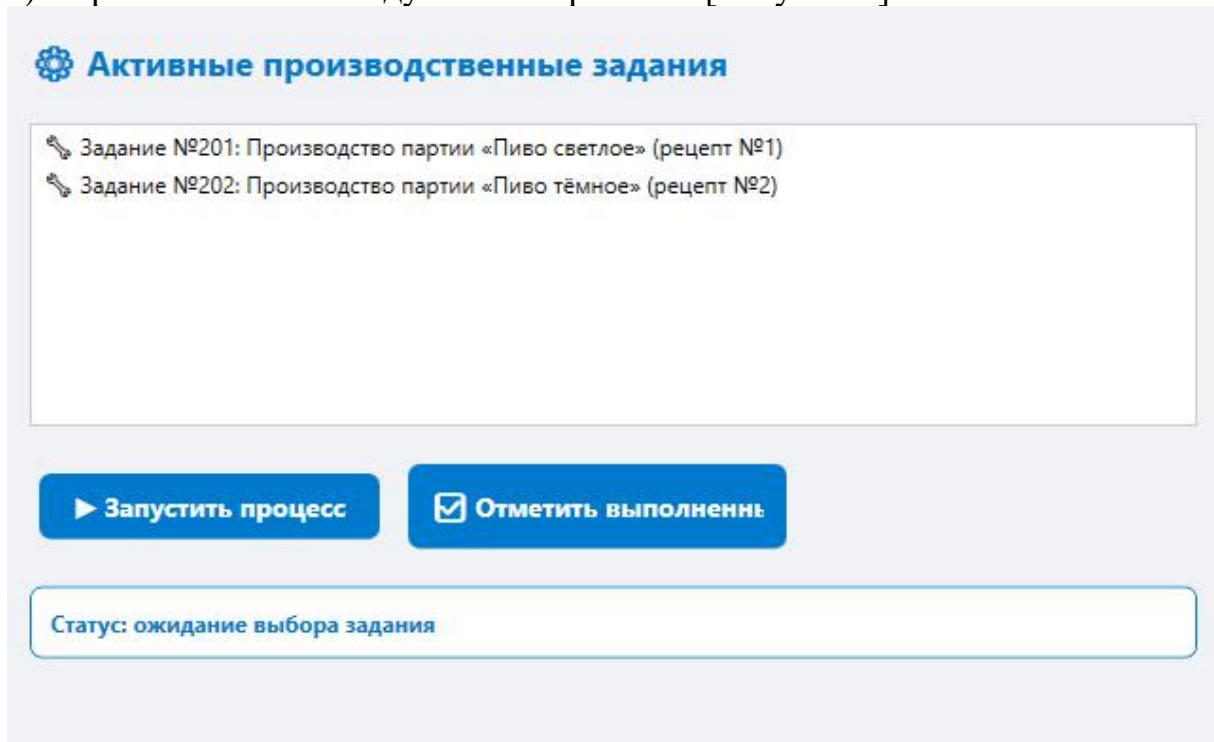


Рисунок 3 - Окно Модуль «Аппаратчик»

2. UML-диаграмма классов (файл diagrams/uml.png в репозитории).
3. ERD-диаграмма базы данных (diagrams/erd.png).
4. Wireframe прототипа интерфейса (diagrams/wireframe.png).
5. Листинг скрипта БД (database/script.sql).
6. Скриншот коллекции Postman (api/postman\_collection.json с запросами).
7. Примеры тест-кейсов и модульных тестов.
8. Ссылка на репозиторий GitHub: <https://github.com/Hefamr01/PM.02-Integration-of-software-modules>